

Judged Whole

One Gate for Untrusted Verification Infrastructure

ANONYMOUS AUTHOR(S)

Verification infrastructure written by a language model cannot be trusted by construction, and neither can a proof it emits. We describe a system in which that is not a problem to be solved but the premise of the design. Every executable — interpreters, translators between languages, searches for evidence, certificate checkers, the maps that carry evidence home — is generated text, and none of it is believed: each enters the system through one gate that runs it against its own shipped cases and its own shipped mutants, and each result the system reports is graded by *where along its route the evidence was judged*. A witness is judged by replaying it through the interpreter of the language the question lives in; a certificate is judged by re-checking it with that language’s checker; a claim with nothing behind it is judged by nothing and says so. The grade is a distance: certified at gap zero, checked at gap k one or more translations away, claimed when no judgment ever ran; and what a result still rests on — its residual — is computed from the log by removing everything upstream of the last judgment, never asserted by the search that found it. The trusted base is exactly the admitted judges, a list the kernel prints. The kernel itself is five small Python modules, generated too and standing outside the gate, so it is made solid another way: the result order and the residual are proved in Lean, the operational half is falsified by its own tests, a second clean-room lineage of its pure half must agree byte for byte, and every admitted entry is pinned by content hash. We report the system exactly as it stands at one frozen commit: two domains (HWMCC’24 bit-vector machines, SV-COMP 2025 C tasks) connected by four generated translation pairs and five generated searches; the boards — 44 of 74, 26 of 79, 47 of 80, 1 of 55 questions settled — with every universal answer’s grade, gap, and residual; the reverse edge whose every route came back a partial; and a ten-minute run that bought one question, showing the boards to be search-limited rather than budget-limited. Honest negatives are reported as results, because that is the only kind of result this system can report.

1 INTRODUCTION

Verification infrastructure is now being written by language models: interpreters for the formats benchmarks arrive in, translators between them, invariant generators, proof scripts, and the checkers that are supposed to catch the rest. Each such artifact is text produced by a process that cannot be inspected and that is wrong at an unknown rate, and the question every deployment faces is what a result computed by such artifacts is worth. Two answers are current. One verifies the artifact: repair the proof until a checker accepts it, or generate the code together with a specification and a proof [14, 36]. The other distrusts the artifact and checks its outputs: certifying algorithms [24], proof-carrying code [28], validated verification witnesses [8]. Both answers are about one artifact at a time. Neither says what happens when the checker is generated too, when the translation that carried the program to the checker is generated, when the map that carries the checker’s verdict back to the original program is generated, and when the search that produced the evidence was generated last week by the same process.

This paper describes a system built on the premise that every one of those pieces is untrusted, and judged whole. It is the sixth generation of hurdy-gurdy, a platform whose earlier generations are described in an arXiv preprint [18] and whose lineage begins with a fixed translator from RISC-V machine code to bit-vector transition systems [19]. The generation described here restarted from an empty repository on 2026-08-20 with a specification and nothing else, and everything that has entered since — every interpreter, translator, search, checker, and carry-back — is generated Python admitted through one gate. The separation that organizes it is a single sentence:

Generation produces syntax; only interpretation produces truth.

A *language* is a syntax with an *interpreter* — the program that runs its programs and reports named observables — and, where the language declares *certificate forms*, a *checker* for each. Interpreters and checkers are the *judges*, and the trusted base of the system is exactly the set of admitted judges, a list the kernel prints. Every other executable is a *transport*: a *translator* from one language to another, a *carry-back* that brings evidence found at the target home to the source, or a *search* that looks for evidence — a *witness* or a *certificate* — within a budget. A transport is untrusted whatever it computes; its output is only ever as good as the judgment it survives. There is exactly one semantic device, the interpreter, and exactly one trust event, an interpreter run judging a transported artifact. **Searches do not decide; they write. Judges decide.**

The consequence for a result is that its *grade* is geometry. A question lives at a language; a *route* carries it across *pairs* of languages to a search; whatever comes back is judged where it lands, and the *gap* of a result is the number of hops between the question and the last judgment its evidence passed. Gap zero is *certified*: a witness replayed by the interpreter of the language the question lives in, or a certificate discharged by that language’s checker. A positive gap is *checked*: the certificate was discharged one or more hops away, and the result still rests on the translators between. No judgment at all is *claimed*. The *residual* of a result — what it still rests on — is computed from the log as the lineages of the hops inside the gap plus the judge that ran; every judgment removes everything upstream of it, so at gap zero the search that found the evidence, and the route it travelled, are not in the residual at all. That *certified* is route-independent is a theorem about the residual, proved in Lean, not a definition.

The kernel that runs the gate, plays routes, keeps the append-only log, and draws the board is five Python modules, generated like everything else. It stands outside the gate because it is the gate, so it is made solid four ways: the half that is mathematics — the result order and the residual — is proved; the half that is operation is falsified by a test suite that re-derives every admission stamp, refuses every shipped mutant, measures the seal, and shows that a lying search, a broken carry-back, a bogus certificate, and a bare claim each lose a grade and never gain one; a second lineage of its pure half, generated clean-room from the specification and the data, is held to byte-agreement with the first; and every admitted entry is pinned by a content hash defined in the specification so that any lineage can re-verify it.

We report the system exactly as it stands at one frozen commit (Section 6). Two domains have entered: hardware, whose root language is BTOR2 [30], and software, whose root is a fragment of C; RISC-V is a third language, anchored by no domain and corroborated only by the squares of the pairs on either side of it. Four generated pairs connect the three languages, five generated searches live at BTOR2 and C, and four benchmarks pinned from HWMCC’24 and SV-COMP 2025 have been played. The boards stand at 44 of 74, 26 of 79, 47 of 80, and 1 of 55 questions settled. Fifteen universal answers on the first board and twelve on the third are certified at gap zero by BTOR2’s own judges, so that the searches that found them are no longer in what they rest on; five on the software board are certified at C, four of them by an interval analysis at C and one by a k-induction certificate found at BTOR2 and carried home. The reverse edge — BTOR2 translated into C so that C’s searches reach hardware questions — was admitted with every channel and returned a partial on every route it opened, for a reason the log states. A ten-minute run of the IC3 search on the ten open universal questions of the first board proved one and was killed at twice the wall on six. The boards are search-limited, not budget-limited, and the paper says so because the system cannot say otherwise.

Contributions.

- A design in which every executable of a verification system is untrusted generated text and one gate admits all of them: the two kinds (language, transport), the three judgments

an artifact faces on crossing a pair (the square, replay, discharge), and the two crossings nothing judges (Section 3).

- Grades as geometry: gap, grade, and residual computed from the log; *certified* as route-independence, with the residual's properties proved (Sections 3.4 and 4).
- Two moves that raise performance without touching trust — *regrade*, which re-discharges a stored certificate closer to home at check cost, and *accelerators*, admitted only by byte-agreement beside a Python reference — and a contract for pairs whose frames do not align (Sections 3.2 and 3.5).
- A kernel outside the gate made solid four ways, each worded no stronger than what it verifies (Section 4).
- Measurements at one frozen commit, with every number tied to a file in the tree, and the negatives reported as results (Section 6).

What is not claimed. The square is checked empirically, per program, on a pair's corpus; per-program translation correctness is not proved. The seal around generated executables is enforced by a blank PATH and an emptied environment, measured by tests, and not claimed to be a proof. No result carries the *corroborated* flag, honestly: every route in the registry shares the C front end and one generator. And the boards are not compared to any solver's on a shared clock; the official verdicts of the competitions are recorded beside each question as testimony, never ranked.

2 ONE QUESTION, TWO ROUTES

Everything the kernel does can be told as the story of one question. Suppose a benchmark pins a C program p — a loop over values read from the outside world, an assertion inside — and asks: can the assertion fail within twenty steps? In this generation that sentence is already formal. C is a *language*: a deterministic syntax plus a generated, deterministic interpreter I_C exposing named *observables* — here bad, whether an assertion failed, and depth, how many statements have run — and the certificate forms it can judge, each with a generated checker. The *question* is p , at C, with its *ask*: the observable bad, the mode exists, the bound 20. It lives at C, and every grade it will ever receive is a distance from that home. The benchmark enters as a *domain*: a *root* language, the one its programs are written in, together with its *anchors* — labels, supplied vectors, the recorded testimony of outside tools — the ungenerable half, and all that entering a new domain costs.

Route one: C to BTOR2. No search reasons about C directly; the model-checking searches live at BTOR2, a language of bit-vector machines. What connects the two is a *pair*: one correspondence, written down as generated code — a translator T that turns the elaborated control flow of p into a machine, one statement one transition, bad a predicate on machine state, and *carry-backs* Λ for whatever will need to come home. Three kinds of artifact cross a pair, and each is judged on arrival by an interpreter run. The program crosses forward, and its judgment is *the square* (Fig. 1), closed for every program of the pair's *corpus* — the programs it ships as its own controls — by running both interpreters: $I_s(p) \equiv_{\pi} \Lambda(I_t(T(p)))$ on the kept observables π . The horizontal arrows are untrusted syntax; the vertical arrows are the judges; nothing horizontal is believed until something vertical has run. A wrong translator — buggy, lazy, adversarial — does not produce wrong results; it loses squares and is refused at the gate.

At the stop. The route ends where evidence is written: a *search*, the one partial transport — budgeted, allowed to return an honest *partial* (how far it got, where it failed), and believed by no one. The searches at BTOR2 are generated solvers: explicit-state enumeration, bounded model checking, k-induction, IC3. Three kinds of *evidence* can come back down the route, and each is judged where it lands. A *witness* — the failing input sequence — is carried back, Λ renaming machine

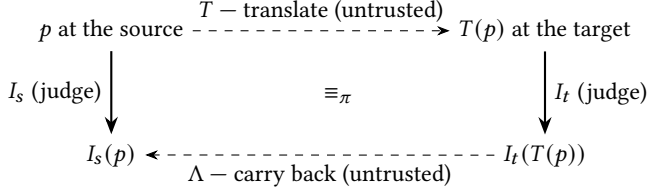


Fig. 1. The square, the judgment on programs. Dashed arrows are transports, generated and untrusted; solid arrows are the two interpreters, the judges. The square is closed empirically, per corpus program, at admission; the observable renaming π is the projection inside it. Where the translation turns one source frame into many target frames, a stimulus map and a bound map ride along and are judged inside the same square against the depths both interpreters report.

stimuli to the input sites of p , and judged by *replay*: I_C runs p on it where the question lives. The gap is zero, and gap zero is *certified*. No generated code can corrupt it: a wrong search, carry-back, or translator all fail the same way, by not producing a stimulus that replays. Witnesses need no declared form, because the judge is the interpreter itself, so a fresh domain certifies existential questions on the day its root interpreter is admitted. A *bare claim* — “no failure within bound k ” with nothing behind it — comes back unjudged, which is exactly why it cannot raise trust; its grade floors at *claimed*. A *certificate* — a k -induction kernel, a clause invariant — comes back and is judged by *discharge*: the checker of the language where it lands re-checks it. Discharged at BTOR2, the claim grades *checked* at gap one, and its *residual* is the lineage of the one hop it has not cleared plus BTOR2’s judge, the search no longer among them. Discharged at C — once the pair learns to carry the certificate itself home — the gap closes and the claim is *certified*. Each judgment removes everything upstream of it from the residual. If a replayed witness ever stands beside a covering universal claim, that is a recorded *contradiction*, never silently resolved: the witness stands, and the universal’s entire chain is marked.

Route two: C to RISC-V to BTOR2. The same question also travels a longer way: compiled to RISC-V — a language whose interpreter is execution of machine code, so reasoning can happen on the executable when that is the faster place — then encoded to BTOR2. Pairs compose into a route; its forward contract is the componentwise meet — fragments intersect, kept observables intersect, bounds cross each hop’s bound map — so a compiler pair that keeps only bad makes the whole route a bad-only route. What the longer route buys is a different *lineage*: its generated descent shares nothing with route one, so agreement between the two would earn the *corroborated* flag, and RISC-V itself, squeezed by squares against anchored neighbours on both sides, is corroborated from both directions at once. A *result* is one play of a route on one question, appended to the run’s log with value, grade, gap, residual, and cost; cost is recorded and never ranked. Three moves raise performance without touching trust: *regrade* — re-discharging a stored certificate closer to home costs check time, not search time; *hints* — seeds a pair passes forward into a search, able to move minutes and not one grade; and *accelerators* — a translator or search generated again in a faster language, admitted only by byte-agreement beside its Python reference, judges always running the reference.

The loop. The player — a language model, or a human at the same command line — plays every question along every admitted route, reads the *frontier* (the questions no result has settled, each with how far every route got), conjectures the next entry in a fixed order (new judging and searching first, then new transports, then new languages), generates it, passes it through the gate, and plays

again, until someone pulls the plug. Pulling the plug is safe at any moment, because the log is append-only and the best result per question only ever improves. The rest of this paper makes each word above exact (Section 3), says how the kernel that does this is itself made trustworthy (Section 4), lists what has been admitted (Section 5), and reports what the plays found (Section 6).

3 THE DESIGN

The specification is written in a vocabulary of fifty words, listed in its last section, and this section uses no others. The registry holds exactly two primitive kinds; everything else — pairs, evidence, grades, routes — is derived from them, and the kernel computes with the derived forms without storing them as kinds.

3.1 Languages, judges, transports, domains

A **language** is a deterministic syntax plus a deterministic interpreter exposing named observables, plus the certificate forms the language can judge, each shipped with a generated checker. Interpreters and checkers are the **judges** — the system’s only semantic devices — and the **trusted base** is exactly the set of admitted judges: a list the kernel prints, each judge with the anchors and controls that corroborate it. A *root* is a language benchmarks arrive in; its interpreter is the trusted floor, corroborated by the domain’s anchors and by nothing generated. Every other language carries no parent taxonomy. Its interpreter is corroborated empirically by the squares of the pairs that connect it, and a language between two anchored neighbours, one square on each side against a different anchored interpreter, is corroborated from both sides at once, which no single pair can give.

A **transport** is a generated function on syntax, declaring its *fragment* — the programs it accepts; outside it, a transport refuses rather than guesses. There are three: the *translator* *T*, a source program to a target program; the *carry-backs* Λ , a target-side artifact to its source-side form — a witness input, a certificate, an observable name; and the *search*, the one partial transport, given a budget and deterministic for that budget, allowed to return *partial*. A route ends at exactly one search. Every transport is untrusted, whatever it computes.

A **domain** is a root language together with its *anchors* — the two things the loop cannot generate for itself: the format questions arrive in, and the ground truth (benchmark labels, supplied vectors, the recorded testimony of outside *oracles*) that corroborates the root’s interpreter. A benchmark lives in exactly one domain, and a domain owns nothing else and fences nothing; every admitted language, pair, and search serves every domain, and the gate, not topical relatedness, is the only membership test.

3.2 Pairs: the three judgments, and two crossings nothing judges

A **pair** is the transports that share one correspondence between a source and a target language — its translator and its carry-backs — declared with a *direction* (exact, over, under), the observables it *keeps*, the artifacts it carries, measured cost, its lineage, and its *corpus*, the programs it ships as its own controls. The grouping is not redundancy: a witness carried back by one translator’s carry-back and replayed against another translator’s output is nonsense; the pair is the statement that its transports are meaningful together. Three kinds of artifact move across a pair, and each is judged on arrival by an interpreter run; there is no other kind of check in the system:

artifact	moves	transport	judged on arrival by
a program	forward	T.py	the square: both interpreters agree on what is kept
a witness	back	lam_wit.py	replay: the source interpreter runs the program on it
a certificate	back	lam_cert.py	discharge: the source language’s checker re-checks it

Two more things cross a pair, and neither is judged. A **bare claim** — “no failure within bound k ” with no certificate behind it — crosses back needing no transport, only the pair’s observable renaming and its *bound cap*; nothing judges it, and that is exactly why it cannot raise trust. A **hint** — seeds, candidates — crosses forward into a search, trust-inert by construction: whatever it seeds is judged on arrival, so a hint can move cost and never a grade.

The square is checked empirically, per corpus program, both interpreters run at admission; it is not a primitive and not a proof, and that remains the platform’s founding economy. Direction declares what may cross back — witnesses along exact and under pairs, universal objects along exact and over — and the square is the empirical evidence for the direction claim. A pair that reifies an unrolling declares a bound cap: an unbounded object crossing it arrives as a bound- k fact.

Frames that do not align. A pair whose translation changes the frame granularity — one source frame becoming many target frames, as one machine step becomes a run of statements — ships two more transports, judged inside the square. A *stimulus map* (`lam_in.py`) carries a source stimulus forward to the one the target interpreter runs on; the square runs the target side on its output, and the witness carry-back must be its inverse on the corpus. A *bound map* (`lam_bound.py`) carries an ask forward and a claim back: forward, the target depth at which a source firing at frame t lands; back, the largest source frame whose image the claim covers, or nothing. Neither is trusted: the gate checks the bound map’s structure — forward strictly monotone with infinity fixed, back its lower adjoint — and holds both maps to the depths the two interpreters report on every corpus stimulus; their mutants must fail there like any other transport’s. The player carries bounds through every hop, books a claim that ends below the first source frame as a partial, and never plays a route that revisits a language, so that a pair in each direction between two languages never plays the round trip.

Certificate forms belong to languages, not to searches. For a certificate to come home, what it looks like must be declared where it lands: a form named at a language and shipped with a generated checker. A pair’s certificate carry-back then maps form to form, per program, through the correspondence its translator already computes.

3.3 Evidence: what a search writes

A search is a transport from a language to evidence about its programs. It declares which observables it can target, it writes, and it pronounces nothing. A **witness** is an input; its judge is the language’s own interpreter, by replay, so no generated code stands between the payload and the trust event. A **certificate** is an object of a form the language declares — a k -induction kernel, a clause invariant, a box per program point — shipped with a checker under the language entry and admitted like everything else: determinism, example certificates that must discharge, mutant certificates that must not. Checkers are judges, part of the trusted base, and deliberately the simplest code in the system; smallness of judges is the honesty metric. A **bare claim** is legal and floors at the lowest grade — intended selection pressure: a search that writes checkable certificates outgrades one that writes bare claims at the same cost, so the ladder itself breeds certificate printers. A **partial** — how far the search got, where it failed — is what a route returns when nothing above came home. A result that decides its question is *settled*: a replayed witness, or a universal claim covering the asked bound. Question modes are not kernel vocabulary: `exists` asks for a witness, `forall` for a universal claim, and a future mode — liveness, resource bounds — is a new certificate form with a new checker, not a kernel change.

3.4 Grades are geometry

Trust is computed, per result, from where its evidence was judged (Fig. 2):

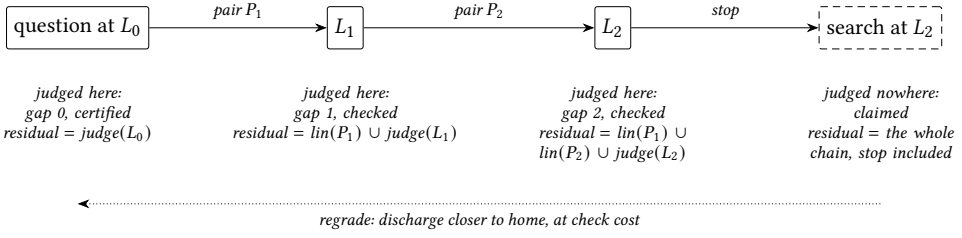


Fig. 2. Grades as geometry. A route of two pairs ends at a search; whatever the search writes is judged where it lands, and the gap counts the hops between the question and the last judgment. The residual is the lineage union over the gap segment plus the judge; every judgment removes everything upstream of it, so at gap 0 nothing but the home judge remains. A bare claim is judged nowhere and rests on the whole chain. Regrade moves a stored certificate down this ladder at check cost.

gap = the hops between the question and the last judgment the evidence passed
grade = *certified* if the gap is 0; *checked* if it is positive; *claimed* if no judgment ever ran
residual = the lineages of the hops still between the question and that judgment, plus the judge that ran

corroborated = results of disjoint lineage agree — an orthogonal flag

Each judgment removes everything upstream of it from the residual. A certificate discharged at the search's language removes the search from what the result rests on — the search may be garbage; the checker validated the object. Carried one hop back and discharged again, it removes that hop too. Discharged where the question lives, it removes the route entirely: *certified* is route-independence, as the theorem $gap = 0$ (Section 4). The fail-safe direction is definitional and universal: transport is untrusted, so a wrong or adversarial carry-back can only lose a grade, never forge one. Witnesses have one property worth stating once: the kernel always replays them where the question lives, chaining carry-backs across the route, so a witness is certified or it is not a result at all; an unreplayable witness is evidence inside a partial, never an answer. A replayed witness is ground truth, and a covering universal claim beside it is a recorded **contradiction**: the witness stands, and the universal's entire chain — evidence, judgments, every transport it crossed — is marked falsified.

Lineage is a declaration about the descent of generated procedures. Two judges grown from the same ancestor never corroborate each other; the corroborated flag requires evidence whose descent sets are disjoint. The second-order effect is intended: whatever earns the top grades is what the loop learns to build, so the frontier itself pushes the generator toward certificate printers, certificate carry-backs, and independently generated judges.

3.5 Routes, results, the order, the frontier

A **question** is a program, the language it lives in, and its *ask*: an observable, a mode, and a bound. A **benchmark** is a pinned finite set of questions with recorded provenance — a sha256 per program, labels where they exist. A **route** is a sequence of pairs ending at one search; its forward contract is the componentwise meet — fragments intersect, kept observables intersect, bounds cross each hop's bound map and bound caps take the minimum, costs add — and how far an artifact travels back is per kind: as far as every pair on the way carries it, its grade computed from where it was last judged, not from what the route promises. A **result** is one play of a route on one question, appended to the log with everything the board needs:

$result = \{ question, route, budget : \{ caps, spent \}, value, grade, gap, residual, lineage \}$

where the value is a replayed witness, a universal claim $all(bound : k \mid \infty, cert?)$ graded by its gap, or a partial with a small typed core the kernel orders on — the bound reached — and free-form progress for the player to read.

Results order per question by one key, stated in the specification exactly because the mechanization and the second lineage of the kernel are both held to it: $(level, bound, grade, gap)$, compared lexicographically, strictly greater is better. Level: 0 a partial, 1 a universal claim below the asked bound, 2 settled. Bound: for a universal claim its bound, ∞ above every number; for a witness, above ∞ itself — a replayed witness stands above any universal claim on its question; for a partial the bound it reached. Grade: ungraded below *claimed* below *checked* below *certified*. Gap: a smaller gap is better, and “no check ever ran” sits below every finite gap. The incumbent survives only while it is strictly better; among records of equal key the latest wins, so the board shows the most recent adjudication of an equally good result. Cost is recorded in every result and never ranked.

Two performance moves are trust-free because they are judged. **Regrade** re-discharges a stored certificate closer to home: it costs check time, not search time, a shrinking gap is a strictly improving result under the order, and a revised judge re-derives a stored certificate’s residual the same way, so a proof is always read under the judges the registry holds now. **Hints** are forward artifacts that can move minutes of search and not one grade.

The **frontier** of a benchmark is the set of questions whose best result is not settled, each carrying that result and its progress. The registry and the log are append-only; best-per-question over an append-only log is monotone: the *ratchet* is a property of the data structure. Expanding the frontier means strictly improving some question’s best result — level, bound, grade, or gap; never cost. The frontier has a second reading, from the registry alone: for every admitted root and every admitted search the kernel can say whether the root reaches it, by which routes, or only for which fragment, or which one missing pair would connect it — a conjecture, never a demand. The frontier is drawn, not only listed: the *board* (one row per question, its best result with value, grade, gap, residual, route, and cost) and the *graph* (languages as nodes, pairs as edges, searches as stops, best results overlaid), both pure functions of the log, both regenerating byte-identically.

The ledger. Cost says what a play spent; the ledger says what it bought, in bits — profiling, never a grade. Witness surprisal $S = -\log_2 \Pr[\text{a random stimulus is a witness}]$, under the uniform measure the interpreter’s havoc rule fixes: every failed random trial tightens a lower bound for free. Cleared bits $B(k)$, the log-size of the stimulus space a bound- k universal claim exhausts, making B per second one clearance currency across every search family. Certificate length L , compressed size under a pinned compressor, making B/L the compression a certificate achieves over exhaustive checking, infinite exactly at bound ∞ . Each pair carries an exchange rate measured on its corpus at admission — how a program dilutes in bytes, how surprisal shifts (exact preserves it, over can only lower it, under only raise it), how a bound rescales. A search may ship a trust-inert ledger.py; what it writes is recorded beside every result and tabled on the board.

3.6 The generation rule: oracles, not organs

Every implementation is generated, in Python, and lives as committed source inside its registry entry. No existing tools: no wrapped engines, no shelling out, no vendored binaries of someone else’s reasoning. The substrate is the Python interpreter the kernel runs on and a declared compiler where an accelerator needs building; nothing that reasons enters except as generated text through the gate. The rule bounds the system, not the evidence about it. At admission time a human or the generator may consult an **oracle** — an existing compiler or solver run entirely outside the sealed runner — and what the oracle says enters the registry the only way ground truth ever enters: as anchors with recorded provenance, corroborating a judge or a transport the way a benchmark

label corroborates a root interpreter. An oracle never enters the trusted base, never runs inside a play, and its disagreement with a judge is a dispute to record and adjudicate, never a verdict. The per-question verdicts a competition archive publishes are anchors of this kind; [Section 6.5](#) reads them beside the boards.

Enforcement is layered, and worded no stronger than what each layer verifies. Statically, there is no manifest field for pointing at a tool. Dynamically, the runner is sealed: every registered executable runs in its own process with an environment emptied to a blank PATH — blank rather than absent, because an unset PATH is replaced by the platform’s default search path, where a tool may sit — in a temporary working directory, under a wall cap, and every check runs twice with byte-compared output. Socially, every implementation is committed source. The seal makes reaching for a tool loud and the registry makes it visible; neither is claimed to be a proof.

Accelerators. Syntax may accelerate; semantics never does. An entry may ship one accelerator — the same implementation generated again in a performance-oriented language, source and built executable both in the entry — only for the play-time transports, *T* and the search, because their outputs face judges downstream. Judges always run the Python reference. An accelerator is admitted solely by byte-agreement with its reference on every admission invocation; the reference remains the semantics. This is where untrusted generation becomes the design’s engine rather than its risk: the architecture maximizes the surface where the generator may be wrong for free — a wrong translator loses squares, a wrong search loses discharge, a wrong carry-back loses replay, none can forge — and the only things needing care are the judges, which the design keeps few, anchored, and mutant-tested.

3.7 The gate

Nothing — no language, pair, search, domain, or any of their implementations, accelerators included — enters the system except by admission, and admission evidence is stamped into the entry by the gate, never self-reported. One discipline for every kind: determinism measured (run twice, byte-compare), the checkable relation run on the entry’s own corpus, and two-sided **controls** — the intact implementation must pass and every supplied **mutant** must fail, because a checker that cannot be made to fail is unfalsifiable. Per kind: a language’s vectors — programs with pinned observables — interpret as expected, and each shipped checker discharges its example certificates and refuses its mutant ones; a pair’s every carried artifact round-trips per corpus program — squares close, stimuli replay, certificates discharge — with mutants per artifact kind; a search’s written evidence on its corpus must judge valid, its negative controls must fail, and its budget determinism must hold; a domain names its root and every anchor’s provenance. An entry that fails leaves no stamp and no trace in routing.

Revision, not mutation. An admitted entry is never edited. Every **stamp** pins the entry’s bytes with a content hash the loader re-verifies, defined in the specification rather than left to an implementation: over every regular file under the entry, excluding the top-level manifest, dotfiles, and bytecode, the map from entry-relative path to the sha256 of the file’s bytes, serialized as JSON with sorted keys, hashed once more. Extension arrives as a new entry <name>@<r> carrying a revision number and its predecessor’s hash. The gate for a revision is the ordinary kind gate plus conservativity: the new implementation must byte-agree with its predecessor on the predecessor’s whole checkable surface — its vectors or corpus, and for a language the corpora of every admitted pair bound to it. A name binds to its highest admitted revision; predecessors stay in the tree; results name the revision they ran. Adding a carry-back to an admitted pair is the intended common case: the old transports are the conserved surface, the new one is gated fresh.

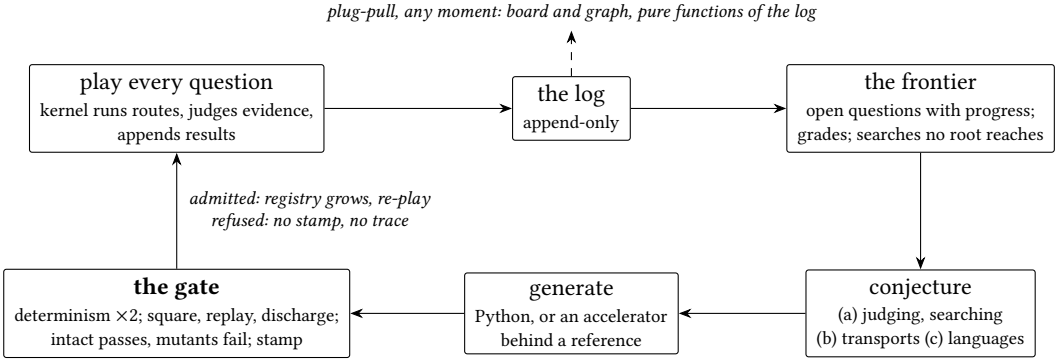


Fig. 3. One loop iteration. The player conjectures and generates; only the kernel writes results, and only the gate writes the registry. Plug-pull is safe at any moment because the exit deliverables regenerate from the append-only log and the best result per question only ever improves.

3.8 The loop, the conjecture order, bootstrap from empty

The player is presented a benchmark and runs until a human pulls the plug (Fig. 3): play — for every question, run the admitted routes within budget, the kernel recording results; read the frontier — the open questions with their progress, and the grades, since a board full of *claimed* and *checked* is itself frontier; conjecture, in the **conjecture order** — (a) new judging and searching for existing languages, (b) new transports, including carry-backs retrofitted onto admitted pairs by revision and a translator from a root that reaches no hub to the nearest one, (c) new languages, proposed only when a translation or solving move keeps winning ad hoc; generate, gate, register; re-play. Semantics first, then syntax: new syntax is earned by demonstrated semantics, never invented ahead of it. The trust story in one sentence: **the generator never writes a result; only the kernel does, by running judges over transported evidence**. Openness is the cheap move: searches and judges live at a few *hub* languages, and a domain enters a hub by one pair — from the moment its root's interpreter and one translator to the hub are admitted, it owns every search there.

The kernel ships with zero languages, zero transports, zero domains. Presented a benchmark, the first admissions are the domain — root name plus anchors, the ungenerable half, nothing executes — then the root language's interpreter, then a first naive search in pure Python. Witnesses need no declared form, so existential questions can settle *certified* with a single admitted interpreter; the first certificate form and checker arrive when universal questions need better than *claimed*. Everything must work at this point: the naive generated search is not a stopgap but the first citizen, and every later move improves on a working, admitted baseline. Section 5 reports that this is how the tree was in fact populated.

4 THE KERNEL, MADE SOLID FOUR WAYS

The kernel is the fixed part: the gate, the sealed runner, the result order, the registry loader, and the driver that plays, regrades, and renders — five stdlib-only Python modules of 2,654 lines at the tag (driver.py 689, gate.py 1,238, registry.py 191, results.py 419, runner.py 85, and a 32-line package file). It is the one piece of code the gate cannot judge, because it is the gate; it is generated like everything else in the tree; and so four disciplines stand in for the gate, each worded no stronger than what it verifies.

It computes no truth of its own. The kernel runs judges, compares bytes, orders results, and draws the log. A kernel bug can lose a result or misgrade one; the property that it can never forge one — no record reaches *certified* unless an interpreter run where the question lives passed, whatever a search, a carry-back, or a translator wrote — is stated in the specification and falsified by test (kernel/tests/test_forge.py). On a registry bootstrapped from empty through the real gate — two toy languages, one pair, one search, one domain — a search that prints a witness together with the words "grade": "certified" never becomes a result, because its witness does not replay; a bogus certificate floors at *claimed*; a bare claim is the checkless crossing; true numbers on an unsafe program do not discharge; a broken carry-back only loses the witness; a route missing a carry-back is a partial, never an answer; an observable the pair does not keep is a partial; and regrade lifts a stored proof to *certified* by check time alone once the pair learns to carry certificates.

The half that is mathematics is proved. The Lean development under kernel/mechanization/[12] — 502 lines, no dependencies beyond the Lean core, toolchain pinned — proves, for exactly the key of Section 3.5, the properties the specification asks for. In Kernel/Key.lean: the order is a strict partial order (Key.lt_irrefl, Key.lt_trans, Key.lt_asymm, axiom-free); best-per-question is monotone under log append — the ratchet (best_mono); once settled, always settled (settled_ratchet); at a fixed level and bound the gap never grows and grades only move up (le_same_value, same_value_ratchet), with grade_moves_key, gap_moves_key, and first_check_moves_key saying that a regrade, which keeps the value, is a strict improvement exactly when it moves the grade or the gap. In Kernel/Trust.lean the residual is defined as the lineage union over the gap segment plus the judge, $\text{residual}(\text{hops}, g, \text{judge}) = \bigcup_{i < g} \text{hops}_i \cup \text{judge}$, and proved to be exactly that meet (mem_residual), from which: gap 0 rests on the judge alone (residual_gap_zero) — *certified* is route-independent as a theorem; a smaller gap never adds trust (residual_mono); every judgment removes everything upstream of it (residual_check_removes_upstream); and the stop is never in the residual (residual_stop_free, residual_le_whole) — once anything checked, the search's own descent is gone. The build prints the axiom audit for every theorem: the order lemmas are axiom-free; the fold lemmas rest on propext and Quot.sound through List.foldl_append, and so do the residual lemmas (residual_gap_zero on propext alone). What the model does not cover is said in its README so the layout never overstates: the proofs are about the order and the residual as functions of keys and lineages, not about the Python that computes them — that correspondence is the job of test_order.py, which runs the same properties against results.key on random records — and the bound is a natural number under an order-embedding of the Python's stand-ins, which lexicographic order cannot distinguish from the original.

The half that is operation is falsified. The test suite (python3 -m kernel.tests; 2,077 lines in eight modules and the toy registry) is the gate turned on the kernel. test_gate runs the gate against the registry's own controls: every bound entry's stamp is re-derived by re-running its admission — its agreement counts a floor, since a stamp is never rewritten and a pair may join after the language it binds to was stamped, an allowance pinned the first time it was needed — and every supplied mutant is refused again; with HG_SLOW=1 every admitted entry of every kind is re-gated, old revisions included; and on the toy registry every way of failing the gate fails it: a language without controls is unfalsifiable, a mutant that passes means the vectors are blind, a judge that accepts anything is refused, reaching for a tool is loud, judges are never accelerated, nondeterminism is refused, a revision must agree with its predecessor, a translator mutant that keeps the square is refused, a lying search is refused because its witness does not replay, a search that only abstains is vacuous, a verdict against a label is refused. test_seal measures the seal: the environment holds nothing but a blank PATH, the working directory is scratch and gone afterwards, no tool can be found — not even on the platform default path — determinism is measured rather

Table 1. The trusted base at the tag, as `python3 -m kernel.driver base` prints it — judges only; everything else in the registry is untrusted syntax. Vectors are programs with pinned observables; controls are mutants the gate refused. For a checker, vectors are example certificates that must discharge and controls mutant certificates that must not. Sizes are lines of Python at the tag.

judge	kind	vectors	controls	lines	corroborated by
btor2@5	interpreter	11	6	390	hardware (3 anchors)
btor2:induction	checker	6	6	1,620	
btor2:clauses	checker	4	4	1,618	
c@5	interpreter	12	6	1,422	software (3 anchors)
c:induction	checker	9	9	4,534	
c:ranges	checker	5	5	3,935	
riscv	interpreter	13	8	603	no domain; its pairs' squares only

than declared, and a hit wall is a result, not an exception. A test that found the seal weaker than the specification claimed — an unset `PATH` still found `/usr/bin/python3` — is how the blank `PATH` got into the specification. `test_registry` recomputes the content pin from its definition for every stamp and refuses changed bytes, pinless stamps, and two entries claiming one name at one revision. `test_misaligned` exercises the frame contract of [Section 3.2](#) end to end, mutants included. `test_regenerate` regenerates the board and the graph of every pinned run byte-identically from the log.

A second lineage agrees. `kernel/second/` is the kernel's pure half — registry loading with pin verification, benchmark pins, the order, best-per-question, the frontier, contradictions, corroboration, the board, the graph, the base; 727 lines — generated clean-room from the specification and the committed data alone, never from the first kernel's source. Its README records the protocol: what was read (the specification, the manifests, the runs, directory listings), what was run (the first kernel only as a black box through its command line; never play, admit, or regrade), one disclosure (an early traceback on a corrupt registry named two source files and line numbers; the first kernel now refuses a corrupt registry without a traceback), and where the specification left it guessing — eleven listed ambiguities on inputs no pinned run exercises. It is held to byte-agreement with the first kernel on base, report, and graph over every pinned run and over a synthetic run holding the corners no pinned run exercises (`test_second`), and a runtime audit hook on open confirms that no module of it reads any file of the first kernel (the import system loads the bytecode of the kernel package's own `__init__`, because `second` is its subpackage, and nothing else). This is the accelerator discipline of [Section 3.6](#) turned on the kernel: two implementations of disjoint descent that agree on every byte corroborate each other, as two judges do. Where they disagreed while being built — the key's tie rule, the pin's exclusions, the binding rule for revisions — the specification was made to say which is right, and now does.

Manual review is the last discipline, and the four above are what make it tractable: what must be read is exactly the trusted base — the admitted judges, printed by `base` ([Section 5](#)), and the kernel's five modules — and the reviewer reads theorem statements, tests, and a second implementation's disagreements rather than a story.

5 WHAT WAS ADMITTED

Everything under `registry/` at the tag entered through the gate of [Section 3.7](#), in the order the conjecture order prescribes, and every entry carries its stamp. [Figure 4](#) draws the registry; [Table 1](#) is the trusted base as the kernel prints it.

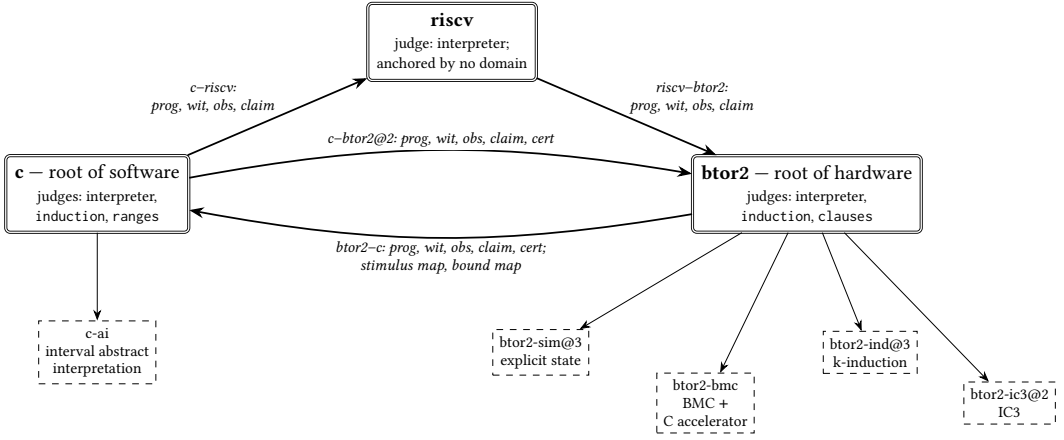


Fig. 4. The registry at the tag: three languages (double borders), four pairs (solid arrows, labelled with the artifacts they carry), five searches (dashed, at the language they live at). Names carry their highest admitted revision. Every node interprets; every edge translates; every search writes. Routes never revisit a language, so the two pairs between c and btor2 never play the round trip.

The first campaign (2026-08-29 and 30). The hardware domain entered first, before its root interpreter existed: root btor2, three anchors — the HWMCC’24 bit-vector track with the official verdicts of the competing solvers as recorded testimony, the pinned benchmark, and eleven hand-written machines with pinned observables. Then the BTOR2 interpreter, the trusted floor: bit-vectors, the signed division family, arrays, constraints; corroborated by nothing generated. Then the naive first citizen, btor2-sim: exhaustive breadth-first enumeration with fixpoint detection when the free-input space is small, deterministic guided sampling otherwise, witnesses only in the second case. Then btor2-bmc: bit-blasting per frame to a hash-consed and-inverter graph, Tseitin encoding, a generated CDCL SAT solver, every model replayed through the interpreter semantics before it is emitted; a bare *all(k)* after unsat at every depth up to *k*; and an accelerator, *solve_fast.c*, mirroring the reference formula by formula, admitted by byte-agreement on the six corpus machines. The software domain and the C interpreter followed — the preprocessor-free ILP32 integer fragment SV-COMP’s loop and bit-vector tasks are written in, semantics total and deterministic and chosen to be exactly mirrorable by a bit-vector machine, validated beyond its twelve vectors by the recorded testimony of `clang -O0 -fwrapv` on random stimuli over the in-fragment corpus (1,045 verdict agreements, none divergent), the compiler never entering. Then RISC-V as the middle vertex, anchored by no domain. Then the three pairs of the triangle — *c-btor2*, *c-riscv*, *riscv-btor2* — each carrying programs, witnesses, observables, and claims, with mutants per artifact kind (translator mutants that lose squares, carry-back mutants that drop values or shift frames) and its exchange rate measured: 3.65, 5.37, and 3.13 target bytes per source byte, as stamped on the revisions that bind today (the first revision of *c-btor2*, on a corpus of seven programs, measured 3.97). Then two certificate forms on BTOR2 by revision of the language — induction (bit-invariants, discharged by three one-step SAT queries for init, consecution, and safety; *k*-induction, by base and step) and clauses (a conjunction of clauses over state bits, as IC3 converges to) — with the two searches that write them, *btor2-ind* and *btor2-ic3*. Then C’s own induction judge, the certificate carry-back of the C-to-BTOR2 pair by revision (*c-btor2@2*), and a further revision of C’s judge that builds its machine by the RISC-V road — front end, fenced RV64 blocks, transition system, sharing no text with the C-to-BTOR2 encoding — so that a proof which

travelled the C-to-BTOR2 pair is certified by an encoding that pair never touched. Last, arrays reached the judges: the bounded checker’s eager reduction, with its congruence and extensionality lemmas, carried into both BTOR2 checkers, C’s judge, and the inductive search.

The reverse edge (2026-09-06 and 07). The C hub gained a third certificate form, ranges — per program point a box over the scalar slots in their own types, unlisted points asserted unreachable, discharged by init, consecution, and safety on the same machine C’s induction judge builds — and its first search, c-ai: interval abstract interpretation over the elaborated control-flow graph with exact wrap images, threshold widening from the program’s own constants, and a narrowing whose every round is accepted only after the recomputed transformer is verified to stay below it; it writes a ranges certificate or an honest partial and never a bare claim. The frame contract of Section 3.2 was written into the specification and its tests into the suite, and on it the pair btor2-c was admitted with all five channels: every state, input, and combinational node a global of the smallest unsigned container holding its width, masked after every operation; one statement per node, i te and every conditional branch-free; a violated constraint clearing a sticky flag; the states assigned together at the end of the loop body, so that a bad at machine frame t is observed at C frame $n_0 + n \cdot t + bad$, the three numbers the translator writes as a header. Its corpus is seventeen machines — the six bit-vector vectors of btor2@5, eight small machines written for the controls, three from hwmcc24-mini; six certificates round-trip (four ranges boxes bit-blasted into clauses over state bits, one k-induction rescaled, one clause invariant carried bit for bit); thirteen mutants are refused (four on programs, two on witnesses, three on certificates, two on the stimulus map, two on the bound map); its exchange rate is 4.19 bytes per byte. Out of revision 1’s fragment, refused loudly: array sorts, widths above 64, non-constant initial values.

Revisions. Eleven language entries, five pair entries, ten search entries, and two domains stand in the tree at the tag; a name binds to its highest revision. The revisions tell the campaign’s small history without editing anything. btor2-sim@2 evaluated the signed division family after four hwmcc24-mini machines crashed the first revision and gained a ledger; @3 scaled its effort by model size after fixed budgets ran past twice the wall on translated C programs. btor2-ind@2 and btor2-ic3@2 blasted the signed division family after thirteen svcomp25-mini programs and four hwmcc24-mini programs crashed the first revisions, and turned every remaining unsupported operation into a partial instead of a crash; btor2-ind@3 ran on the bounded checker’s array engine. Each was byte-identical to its predecessor on the predecessor’s corpus, which is what lets the stamps that depend on it keep their meaning.

6 MEASUREMENTS

Every number in this section is read from the tree at tag era6-campaign-2: the boards runs/*/frontier.md, which regenerate byte-identically from the logs runs/*/log.jsonl; the pinned benchmarks runs/*/benchmark.json, whose official field records per question the verdict the competition archive published; and the stamps in the registry. Nothing is reported that is not in the log.

Protocol. Four benchmarks are pinned, every program by sha256 (Table 2). Three are tiers of the HWMCC’24 bit-vector track [16, 30] selected in the fifth generation and carried forward: hwmcc24-mini (74 machines, 23% of the track), hwmcc24-arrays (55 machines with array sorts), and hwmcc24-mid (80). The fourth, svcomp25-mini, holds 79 ReachSafety loop and bit-vector tasks from SV-COMP 2025 [5], 31 of them tasks that at most three of the verifiers in the archive answered correctly. An ask is exists bad at a bound, or forall bad at a bound or at ∞ . A play runs every question along every admitted route whose fragment admits it, one wall per route: 60 s on hardware

Table 2. The pinned benchmarks and the boards at the tag. Settled: a replayed witness, or a universal claim covering the ask. Certified universal answers are at gap 0; every witness is certified by replay at the root; covering claims are bare, graded *claimed*. Official verdicts are read from `benchmark.json`; unsolved means no official solver answered.

benchmark	questions	settled	of which			open	open questions, officially	
			cert. $\forall$	witness	claim		unsolved	solved
hwmcc24-mini	74	44	15	9	20	30	7	23 (17 unsat, 6 sat)
svcomp25-mini	79	26	5	14	7	53	0	53 (30 true, 23 false)
hwmcc24-mid	80	47	12	8	27	33	8	25 (17 unsat, 8 sat)
hwmcc24-arrays	55	1	0	1	0	54	38	16 (3 unsat, 13 sat)

throughout; on software 30 s for the first campaign’s full plays (its very first play, over the fourteen hardest tasks alone, ran at 60 s) and 60 s since; a search that runs to twice its wall is killed and booked as a partial saying so. The player is the driver’s play; later passes used `-via` (only the routes an entry opens) and `-only` (a named subset of questions), alone or together, and the log records which.

6.1 The first campaign (2026-08-30)

The hardware board closed at 43 of 74 after four plays (the fourth, with every search at its revised entry, moved nothing): 14 universal answers *certified* at gap 0 — thirteen induction certificates written by `btor2-ind` and one clause invariant written by `btor2-ic3`, each discharged by BTOR2’s own judge, so that the search that found it is no longer in its residual — 9 witnesses certified by replay (four found by bounded model checking, three by the naive enumeration, two by the inductive search’s bounded phase), and 20 bounded claims covering their asks, two of them *all*(∞) bare claims from the naive search’s enumeration to a fixpoint, graded *claimed* because a fixpoint the search reports is not a certificate the judge can check. The residual of every certified answer on the board reads `btor2-cert-g1` `btor2-interp-g1` and the two generator tags — the judge and nothing else.

The software board closed at 26 of 79: 14 witnesses *certified* at C, twelve of which walked home across both bridges of the triangle — found at BTOR2 on the route C to RISC-V to BTOR2 and replayed by I_C — and two across C to BTOR2 with the naive search; three universal answers first *checked* at gap 1 (k-induction certificates with $k = 7, 6, 3$, discharged by BTOR2’s judge, their residual naming the C-to-BTOR2 pair) and then, once C shipped its own induction checker and the pair gained a certificate carry-back by revision, lifted to *certified* at gap 0 by regrade alone, without re-solving anything, in 0.55, 0.41, and 0.51 s; then, when C’s judge was revised to build its machine by the RISC-V road, re-judged again by regrade (0.82, 0.42, 0.58 s) so that their residual no longer names the route the evidence travelled; then a fourth proof, about a C program with a 2048-element array, found k-inductive at BTOR2, discharged there, carried home, and certified at C with every array a state of its own; and eight bounded claims covering an ask of 100. No contradiction anywhere. No *corroborated* flag either, and honestly so: every route shares the C front end and one generator, and the flag is reserved for disjoint descent.

6.2 The reverse edge (2026-09-07)

The pair `btor2-c` of Section 5 opens one new route on every hardware question: BTOR2 to C to `c-ai`. Of `hwmcc24-mini`’s 74 machines, 57 translate and agree with BTOR2’s own interpreter on random stimuli; 17 are refused at widths above 64, revision 1’s fragment. Played through the new

route alone, every one of the 74 routes came back a partial, and the log says why: on 41 machines, intervals see ERR reachable, because the pinned branch-free $i \oplus e$ encoding — $b \oplus ((a \oplus b) \wedge -c)$ — is opaque to a box; on 4, the iteration budget was spent before a fixpoint; 12 are existential questions, which a proving search never refutes; 17 are the refused translations. On `svcomp25-mini`, `c-ai` certified four universal answers at gap 0 in 0.038 s of search each — `safe1`, `safe6`, and `safe21`, already certified by the BTOR2 route and now shown under `c-ai` because the key is equal and the latest adjudication wins, and `safe14`, previously settled by a bare claim covering its ask of 100 and now *all(∞) certified* — and returned a partial on the other 75: 38 with ERR reachable in the abstraction, 37 existential. **Both boards stayed where they were: 43 of 74 and 26 of 79.** That is the measurement the edge was built to make: the bridge is judged and open in both directions, and what a word-level search buys on hardware questions is bounded by what it can see through masks. The next search at C moves the board, or does not, on an edge that no longer needs building.

6.3 The two pinned rungs, played for the first time (2026-09-07)

`hwmcc24-arrays` settled 1 of 55: one witness at depth 6, found by bounded model checking in 0.5 s and again by the inductive search, certified by replay; and 99 bounded claims (50 by `btor2-bmc`, 49 by `btor2-ind`) that cover none of their asks — 38 of the 55 questions ask $\forall$ at ∞ , and the deepest bound reached is 66. The reason is structural, and the log names it on every row: the explicit-state search and IC3 refuse array sorts (55 partials each, unsupported `sort: array`); the reverse edge refuses all 55; four machines with nested arrays lie outside the bounded searches' fragment and are refused by both; and one inductive run was killed at twice the wall. On this rung only bounded model checking and k-induction run at all, and k-induction found no inductive strengthening on any of the 49 machines it ran to a bound on. Officially, 38 of the 54 open questions were solved by none of the five solvers in the archive; the other 16 were.

`hwmcc24-mid` settled 47 of 80: 12 universal answers *certified* at gap 0 by the inductive search, 8 witnesses certified by replay (six by bounded model checking, one by enumeration, one by the inductive search), and 27 bounded claims covering their asks (21 by k-induction's bounded phase, 6 by bounded model checking): 26 answer $\forall$ at 20, and one, `unsafe3`, answers an existential ask at 30 in the negative — no failure within 30 frames, three searches agreeing, nothing judging them. Of the 33 open, the log's partials say what was spent: the explicit-state search reports free bits exceed enumeration on 79 of 80 machines; the interval route spent its iteration budget before a fixpoint on 35 and was refused translation on 23; 21 runs were killed at twice the wall (14 inductive, 4 interval, 2 bounded, 1 IC3); and six runs — three inductive, three IC3 — ended in a search failure with exit code 1, booked as partials. Officially, the 8 open universal questions at ∞ were solved by none of the eight solvers; the 17 open bounded and 8 open existential questions were.

6.4 Ten minutes of IC3 (2026-09-07)

The ten open universal questions of `hwmcc24-mini` — seven officially unsolved, three officially unsat at ∞ — were played through `btor2-ic3` alone with a wall of 600 s (Table 3). One proof: `open5`, a 64-bit machine officially unsat for all eight solvers within 4 s, proved by a clause invariant in 30.2 s once the budget let the search go deep enough, discharged by `btor2:clauses`, *certified* at gap 0. Three bounded claims written, of which one enlarged the board's best bound (`open6`, from 133 to 177) and two matched the incumbent's 300. Six runs killed at twice the wall with nothing to show. `hwmcc24-mini` stands at 44 of 74. Ten times the clock bought one question: the board is search-limited, not budget-limited, and what moves it next is a search that reasons differently, not a longer wall.

Table 3. The ten-minute pass: btor2-ic3@2 at wall 600 s on the ten open universal questions of hwmcc24-mini, read from iteration 5 of the log. Best before is the board’s best bound after the 60 s plays.

question	officially	best before	result	spent (s)
open5	unsat, 8 of 8 solvers	<i>all</i> (1)	<i>all</i> (∞), <i>certified</i> , gap 0	30.2
open6	unsolved	<i>all</i> (133)	<i>all</i> (177), <i>claimed</i>	173.7
open3	unsolved	<i>all</i> (300)	<i>all</i> (300), <i>claimed</i>	128.4
safe5	unsat, 7 solvers	<i>all</i> (300)	<i>all</i> (300), <i>claimed</i>	168.8
open2	unsolved	<i>all</i> (300)	killed at twice the wall	1210.5
open7	unsolved	<i>all</i> (3)	killed at twice the wall	1210.4
open8	unsolved	<i>all</i> (2)	killed at twice the wall	1210.3
open9	unsolved	<i>all</i> (3)	killed at twice the wall	1210.7
open10	unsolved	<i>all</i> (2)	killed at twice the wall	1210.1
safe6	unsat, 2 solvers	<i>all</i> (239)	killed at twice the wall	1210.9

6.5 Against the official verdicts

The competitions’ verdicts are anchors: recorded per question with provenance, drawn beside the board, never ranked, never a judge. Read that way, the four boards say the following. On hwmcc24-mini, all 44 settled questions carry an official verdict that agrees with the board’s; of the 30 open, 7 were solved by no official solver and 23 were — 15 bounded safety questions on which the bounded searches reach depths of 2 to 17 against an ask of 20, two safety questions at ∞ bounded at 300 and 239, and 6 existential questions at bound 30 whose witnesses the searches did not find. On svcomp25-mini, every one of the 79 tasks was solved by at least one verifier, and every one of the 26 settled here agrees with its label; the 53 open include all 31 tasks selected because at most three verifiers answered them, none of which any route here settles — the bounded searches reach depths of 11 to 300 on them, all below ∞ . On hwmcc24-mid, the 8 open universal questions at ∞ are exactly the 8 no official solver answered; the one existential question settled in the negative, unsafe3, is recorded sat by five solvers at a depth the archive does not publish — a bounded bare claim beside an unbounded verdict, which is not a contradiction and is the one settled row on any board whose official verdict is not the same answer. On hwmcc24-arrays, the one settled question agrees with its label, and 38 of the 54 open were solved by no official solver. No board holds a replayed witness or a discharged certificate whose official verdict disagrees with it, and no board holds a question settled here that the archive records as unsolved.

6.6 What the numbers say

Three readings are supported by the log, and no more. First, the geometry does what it was built to do: 32 universal answers across the boards are *certified* at gap 0, and for every one of them the residual printed on the board is the home judge and nothing else — the search, and on the software board the route, are gone from what the answer rests on; the regrade of three C proofs moved them from gap 1 to gap 0 in under a second each without re-solving, and re-judged them again under a judge of different descent. Second, the honesty rules bite where they should: the 20, 7, and 27 covering claims on the three boards are *claimed*, and the two *all*(∞) answers from the naive search are *claimed* despite being the strongest value on their questions, because nothing judged them. Third, the negatives are structural, not budgetary: the reverse edge is blind to the *ite* idiom, the array rung is refused by three of five searches, and ten minutes of IC3 bought one question. The next moves the tree names — a property-directed search with proper generalization at BTOR2,

an explicit-state search that accepts arrays, an interval analysis that reads the `ite` idiom — are (a)-moves of the conjecture order, and the boards will say whether they move.

7 RELATED WORK

Generated verification infrastructure. Language models are used to write proofs and repair them until a checker accepts them [14], to propose invariants for a verifier to confirm [31, 41], to synthesize code together with its specification and proof [26], and to keep code, documentation, and tests consistent under a checker’s eye [36]. In every case the checker is fixed and trusted and the model fills one slot. The system here lets the model fill every slot — interpreter, translator, checker, search, carry-back — and moves trust to a property no slot can assert: where along a route an interpreter run judged the evidence. The instrument paper of the same project [18] gave the calculus of directional squares, the componentwise meet, and the replay asymmetry that this generation consumes; its measured claims are about an earlier design in which engines were wrapped and registration was human, and it is cited here as prior work, not restated.

Translation validation and verified compilation. Checking a translation per program rather than proving the translator [29, 32] is the square of Section 3.2, with two differences: both sides are generated interpreters rather than a trusted semantics, and the check is on a shipped corpus at admission with mutants that must fail, not on every compilation. Verified compilers [21, 22] and verified translation validators [34, 39] buy a stronger statement at the cost of a proof per translator; Alive2 [23] bounds the check the way a bound cap bounds a claim here. The system does not prove any translator and says so; what it adds is that the grade of a result never depends on whether a translator was right, only on whether a judge ran.

Certificates, witnesses, and proof formats. Certifying model checkers [27], certified hardware model checking [43], verification witnesses [7, 8], proof-carrying code [28], certifying algorithms [24], and checked proof formats [1, 37, 40] share the move that an untrusted producer emits an object a small checker validates. Here the checker is generated too, admitted by its own controls, and the object’s grade records how many translations lie between the checker that validated it and the question; the absence of a certificate is also data, a *claimed* result being a standing demand for the printer that would upgrade it. Certified abstract interpretation [4, 17] produces or verifies the kind of certificate `c`: ranges checks; the difference is that the analysis here is untrusted and the checker is what was gated.

The searches. The generated searches implement known algorithms: bounded model checking [9], k-induction [35], IC3 and property-directed reachability [10, 13], interval abstract interpretation [11]; none is claimed to be competitive with the solvers whose verdicts sit beside the boards. Btor2C [6] translates BTOR2 machines to C so that software verifiers reach hardware questions; the reverse edge of Section 6.2 is the same move, with the translation gated by squares in both directions and its bound map judged against both interpreters — and its result at the tag is a negative.

The trusted base. That trust cannot be bootstrapped from a component one did not judge is old [38]; the LCF discipline of a small kernel every proof must pass [15] is the shape of the trusted base here, with the kernel’s own standing established by proof, tests, a second lineage, and pins rather than by being small enough to believe. Differential testing [25, 42] and N-version programming [3] supply the second-lineage discipline, and the finding that independently written versions fail together [20] is why the corroborated flag requires declared disjoint descent and is, at the tag, never earned. An evidential tool bus [33] records the evidence tools produce; here the tools are not trusted to produce it, and what is recorded is where an interpreter judged it. Rotor [19], from which the project descends, fixed one translation from RISC-V to BTOR2; the pairs `c-riscv` and

riscv-btor2 are that translation generated and gated, with the ISA semantics generated rather than taken from a specification language [2].

8 LIMITATIONS AND CONCLUSION

The limits are the ones the tree states. The square is empirical: a pair is admitted when its squares close on its corpus, and a corpus program is not every program; the mutants make the corpus falsifiable, not complete. The seal is a blank PATH and an emptied environment, measured by tests; an absolute path to a tool is hidden by nothing but the committed source that would show it. The corroborated flag has never fired, because every route shares the C front end and one generator, and the second BTOR2 lineage the fifth generation had — a BDD search written from the format’s semantics alone — is not regenerated. The judges are not all small: BTOR2’s are 390, 1,618, and 1,620 lines, but C’s induction and ranges checkers are 4,534 and 3,935 lines, because each embeds the front end, the RISC-V-road encoding, and the bit-blaster it judges with; the honesty metric of Section 3.3 says so rather than hiding it. The reach table, the oracle column, and the per-domain ledger are specified and not drawn; no domain records performance testimony, so the boards stand beside the archives’ verdicts and not their times. And the fifth generation’s boards on the same three hardware benchmarks — 46 of 74, 5 of 55, 45 of 80, under a trust story in which searches pronounced verdicts on their own admitted word and with a BDD search this generation has not regenerated — were higher on two of the three; that is what removing the seam and refusing arrays at three of five searches cost, recorded in the history and not rounded away.

Within those limits the claim is exact in kind. A verification system can be built entirely from generated, untrusted text — interpreters, translators, searches, checkers, carry-backs, and the kernel itself — and judged whole: one gate admits everything by running it against its own controls; one trust event, an interpreter run, is the only source of truth; a result’s grade is the distance from the question to the last such run, and its residual is computed by removing everything upstream of that run; and the kernel that does this is proved where it is mathematics, falsified where it is operation, doubled by a lineage that shares nothing with it, and pinned. At one frozen commit that system holds four boards whose every judged answer agrees with the competitions’ verdicts, 32 universal answers that rest on a home judge alone, and three honest negatives — an edge that returned only partials, a rung refused by three of five searches, and ten minutes that bought one question — reported as results, because the design leaves no other way to report them. The generator never writes a result; only the kernel does, by running judges over transported evidence. What moves the boards next is a search that reasons differently, admitted through the same gate, and the boards will say whether it did.

REFERENCES

- [1] Bruno Andreotti, Hanna Lachnitt, and Haniel Barbosa. 2023. Carcara: An Efficient Proof Checker and Elaborator for SMT Proofs in the Alethe Format. In *TACAS*.
- [2] Alasdair Armstrong, Thomas Bauereiss, Brian Campbell, Alastair Reid, Kathryn E. Gray, Robert M. Norton, Prashanth Mundkur, Mark Wassell, Jon French, Christopher Pulte, Shaked Flur, Ian Stark, Neel Krishnaswami, and Peter Sewell. 2019. ISA Semantics for ARMv8-A, RISC-V, and CHERI-MIPS. *Proceedings of the ACM on Programming Languages* 3, POPL (2019), 71:1–71:31.
- [3] Algirdas Avizienis. 1985. The N-Version Approach to Fault-Tolerant Software. *IEEE Transactions on Software Engineering* SE-11, 12 (1985), 1491–1501.
- [4] Frédéric Besson, Thomas Jensen, and David Pichardie. 2006. Proof-Carrying Code from Certified Abstract Interpretation and Fixpoint Compression. *Theoretical Computer Science* 364, 3 (2006), 273–291.
- [5] Dirk Beyer. 2021. Software Verification: 10th Comparative Evaluation (SV-COMP 2021). In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS, Vol. 12652)*. 401–422.
- [6] Dirk Beyer, Po-Chun Chien, and Nian-Ze Lee. 2023. Bridging Hardware and Software Analysis with Btor2C: A Word-Level-Transition-System-to-C Translator. In *TACAS (LNCS)*.

- [7] Dirk Beyer, Matthias Dangel, Daniel Dietsch, Matthias Heizmann, Thomas Lemberger, and Michael Tautschnig. 2022. Verification Witnesses. *ACM Transactions on Software Engineering and Methodology* 31, 4 (2022).
- [8] Dirk Beyer, Matthias Dangel, Daniel Dietsch, Matthias Heizmann, and Andreas Stahlbauer. 2015. Witness Validation and Stepwise Testification across Software Verifiers. In *ESEC/FSE*.
- [9] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. 1999. Symbolic Model Checking without BDDs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS, Vol. 1579)*. 193–207.
- [10] Aaron R. Bradley. 2011. SAT-Based Model Checking without Unrolling. In *VMCAI (LNCS, Vol. 6538)*. 70–87.
- [11] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *POPL*. 238–252.
- [12] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *CADE (LNCS, Vol. 12699)*. 625–635.
- [13] Niklas Eén, Alan Mishchenko, and Robert Brayton. 2011. Efficient Implementation of Property Directed Reachability. In *FMCAD*. 125–134.
- [14] Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. 2023. Baldur: Whole-Proof Generation and Repair with Large Language Models. In *ESEC/FSE*.
- [15] Michael J. C. Gordon, Robin Milner, and Christopher P. Wadsworth. 1979. *Edinburgh LCF: A Mechanised Logic of Computation*. LNCS, Vol. 78. Springer.
- [16] HWMCC. 2024. Hardware Model Checking Competition 2024. <https://hwmcc.github.io/2024/>. Benchmarks and results archive.
- [17] Jacques-Henri Jourdan, Vincent Laporte, Sandrine Blazy, Xavier Leroy, and David Pichardie. 2015. A Formally-Verified C Static Analyzer. In *POPL*. 247–259.
- [18] Christoph Kirsch. 2026. Untrusted Authors, Trusted Answers: A Calculus of Fidelity-Graded Translations. arXiv preprint arXiv:2607.14137, v2.
- [19] Christoph M. Kirsch. 2017. Selfie and the Basics. In *Onward!*
- [20] John C. Knight and Nancy G. Leveson. 1986. An Experimental Evaluation of the Assumption of Independence in Multiversion Programming. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 96–109.
- [21] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. 2014. CakeML: A Verified Implementation of ML. In *POPL*.
- [22] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* 52, 7 (2009), 107–115.
- [23] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: Bounded Translation Validation for LLVM. In *PLDI*.
- [24] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161.
- [25] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10, 1 (1998), 100–107.
- [26] Md Rakib Hossain Misu, Cristina V. Lopes, Iris Ma, and James Noble. 2024. Towards AI-Assisted Synthesis of Verified Dafny Methods. *Proceedings of the ACM on Software Engineering* 1, FSE (2024).
- [27] Kedar S. Namjoshi. 2001. Certifying Model Checkers. In *CAV*.
- [28] George C. Necula. 1997. Proof-Carrying Code. In *POPL*.
- [29] George C. Necula. 2000. Translation Validation for an Optimizing Compiler. In *PLDI*.
- [30] Aina Niemetz, Mathias Preiner, Clifford Wolf, and Armin Biere. 2018. Btor2, BtorMC and Boolector 3.0. In *CAV*.
- [31] Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2023. Can Large Language Models Reason about Program Invariants?. In *ICML*.
- [32] Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. Translation Validation. In *TACAS*.
- [33] John Rushby. 2005. An Evidential Tool Bus. In *ICFEM*.
- [34] Thomas Arthur Leck Sewell, Magnus O. Myreen, and Gerwin Klein. 2013. Translation Validation for a Verified OS Kernel. In *PLDI*.
- [35] Mary Sheeran, Satnam Singh, and Gunnar Stålmarck. 2000. Checking Safety Properties Using Induction and a SAT-Solver. In *FMCAD (LNCS, Vol. 1954)*. 108–125.
- [36] Chuyue Sun, Ying Sheng, Oded Padon, and Clark Barrett. 2024. Clover: Closed-Loop Verifiable Code Generation. *arXiv preprint arXiv:2310.17807* (2024).
- [37] Yong Kiam Tan, Marijn J. H. Heule, and Magnus O. Myreen. 2021. cake_lpr: Verified Propagation Redundancy Checking in CakeML. In *TACAS*.
- [38] Ken Thompson. 1984. Reflections on Trusting Trust. *Commun. ACM* 27, 8 (1984), 761–763.
- [39] Jean-Baptiste Tristan and Xavier Leroy. 2008. Formal Verification of Translation Validators: A Case Study on Instruction Scheduling Optimizations. In *POPL*.
- [40] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt Jr. 2014. DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs. In *SAT*.

[41] Haoze Wu, Clark Barrett, and Nina Narodytska. 2024. Lemur: Integrating Large Language Models in Automated Program Verification. In *ICLR*.

[42] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *PLDI*.

[43] Emily Yu, Armin Biere, and Keijo Heljanko. 2021. Progress in Certifying Hardware Model Checking Results. In *CAV*.